Homework 1: Gradient Descent

```
import numpy as np
import matplotlib.pyplot as plt
```

Exercise 1: GD on a 1D Function

Consider the 1-dimensional function

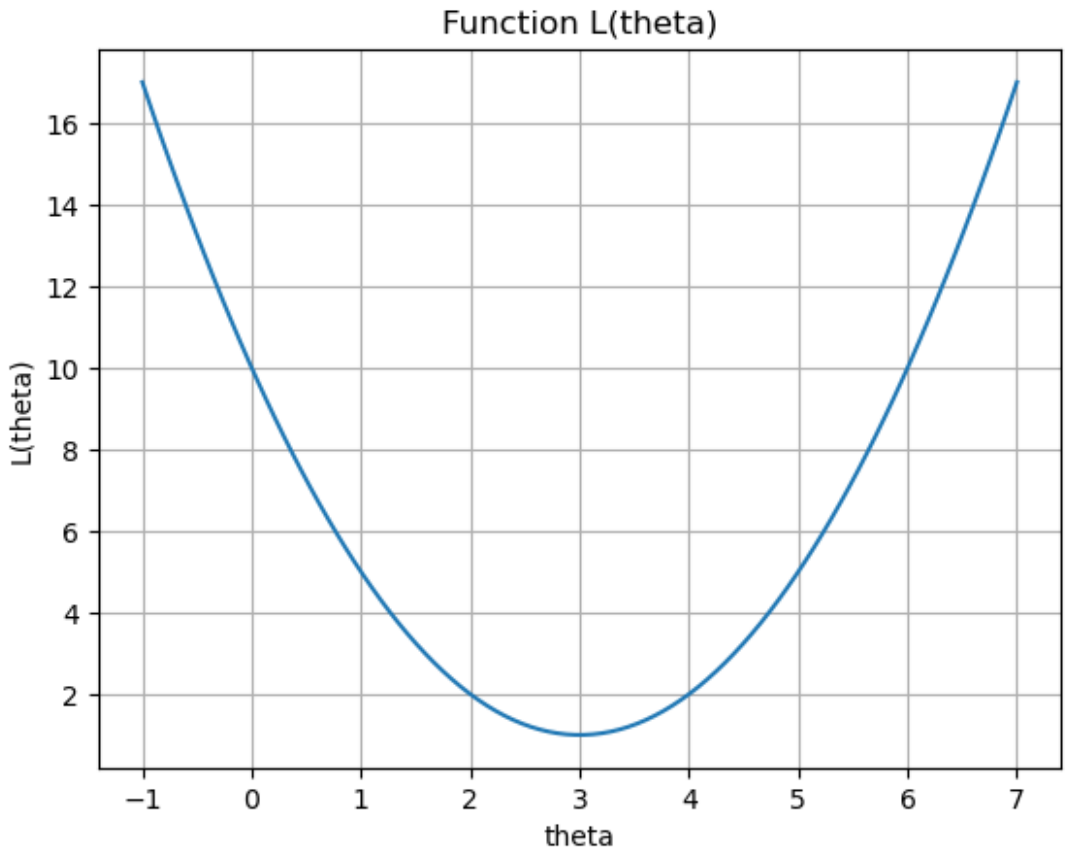
$ L(\theta)= (\theta-3)^2 + 1 $

1.1 Compute the Gradient of $ L(\theta) $ explicitly.

```
# One-dimensional function
theta_0 = 1.0

def l(theta):
    return (theta -3)**2 + 1

def grad_l(theta):
    return 2*(theta -3)
```

Hint: This function is strictly convex with a unique minimizer at $ \theta^{*} = 3 $

```
# Draw L(theta)
theta_vals = np.linspace(-1, 7, 100)
L_vals = l(theta_vals)
plt.plot(theta_vals, L_vals)
plt.xlabel('theta')
plt.ylabel('L(theta)')
plt.title('Function L(theta)')
plt.grid()
plt.show()
```



1.2 Implement the Gradient Descent to optimize $ L(\theta) $ following what we introduced on the theoretical sections.

```
def GD(l, grad_l, theta_0, maxit, eta, tolL, tolTheta):
    loss_history = []
    theta_history = []

    for k in range(maxit):
        theta = theta_0 - eta * grad_l(theta_0)

        if(np.linalg.norm(grad_l(theta))<tolL or
(np.linalg.norm(theta-theta_0)<tolTheta)):
            break

        loss_history.append(l(theta))
        theta_history.append(theta_0)

        theta_0 = theta
    return theta, k, loss_history, theta_history

maxit = 100
tolL = 1e-6
tolTheta = 1e-6
eta = 0.1
```

```
theta_opt, num_iter, _, _ = GD(l, grad_l, theta_0, maxit, eta, tolL,
tolTheta)
print("Optimal theta:", theta_opt, " iterations:" , num_iter)
print("Optimal value of L:", l(theta_opt), " value of L at theta0:",
l(theta_0))
```

```
Optimal theta: 2.9999961687611476  iterations: 58
Optimal value of L: 1.0000000000146785  value of L at theta0: 5.0
```

1.3 Test three different constant step sizes:

- $ \eta = 0.05 $
- $ \eta = 0.2 $
- $ \eta = 1.0 $

```
etas = np.array([0.01, 0.2, 1.0])
loss_histories = []
theta_histories = []

for eta in etas:
    theta_0 = 1.0
    theta_opt, num_iter, loss_history, theta_history = GD(l, grad_l,
theta_0, maxit, eta, tolL, tolTheta)
    loss_histories.append(loss_history)
    theta_histories.append(theta_history)
    print(f"Eta: {eta} => Optimal theta: {theta_opt}, iterations:
{num_iter}, L(theta_opt): {l(theta_opt)}")
```

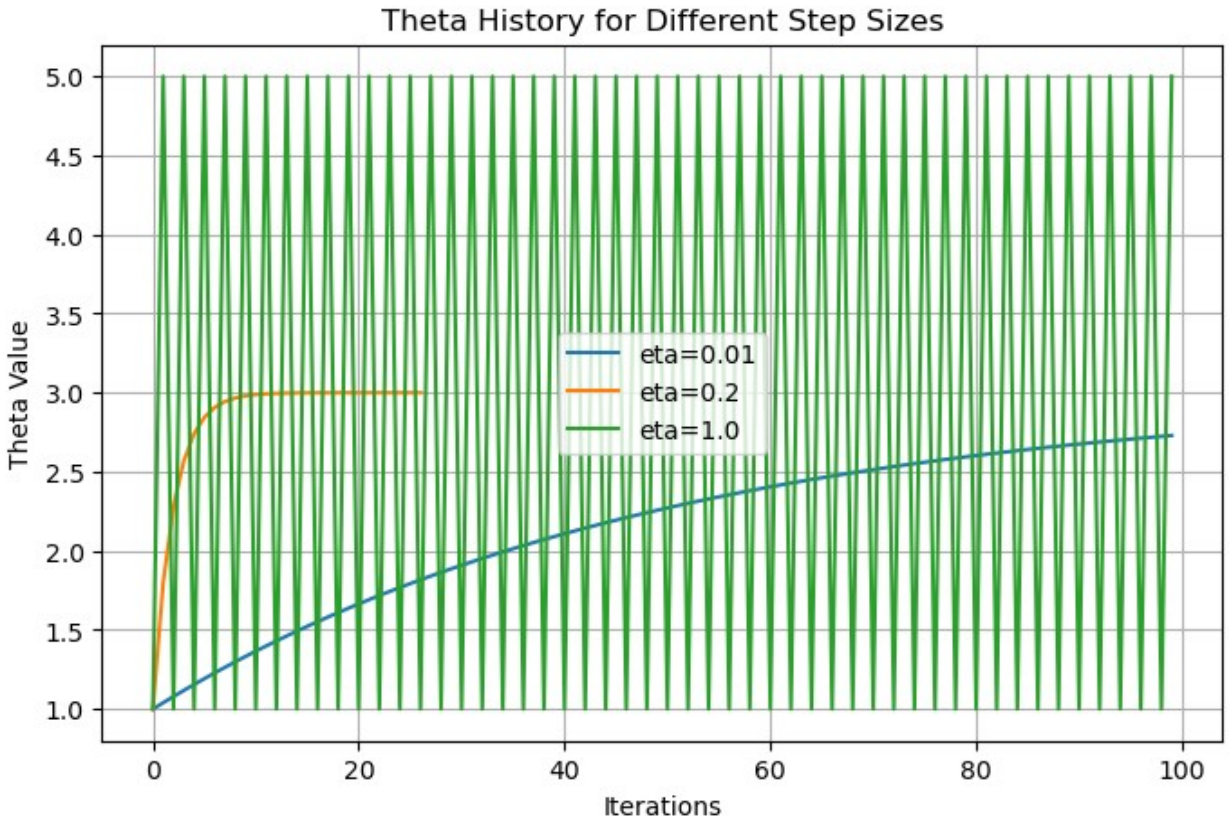
```
Eta: 0.01 => Optimal theta: 2.7347608882104932, iterations: 99,
L(theta_opt): 1.0703517864228864
Eta: 0.2 => Optimal theta: 2.9999987718115575, iterations: 27,
L(theta_opt): 1.0000000000015083
Eta: 1.0 => Optimal theta: 1.0, iterations: 99, L(theta_opt): 5.0
```

1.4 For each choice:

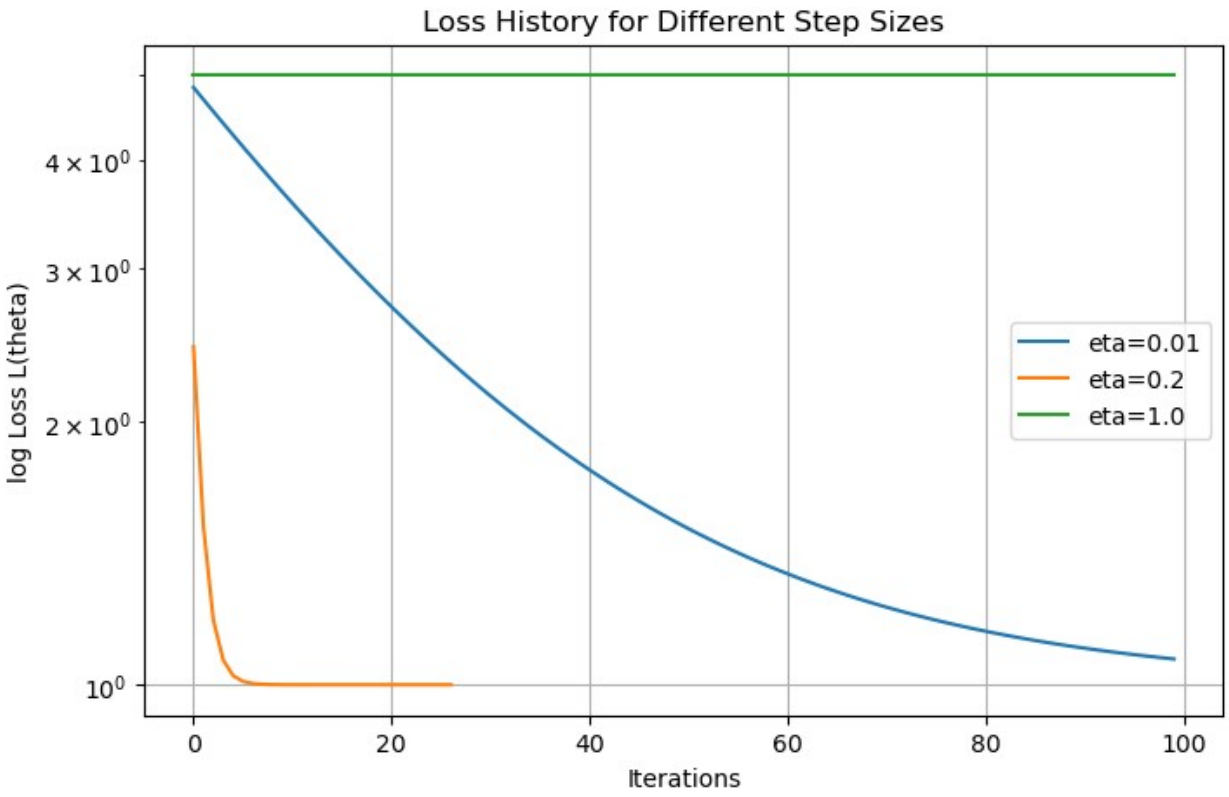
- Plot the sequence $ \theta^{k} $ on the real line (i.e. draw a line and represent, on top of it, the position of all the successive elements $ \theta^{k} $).
- Plot the function values $ L(\theta^{k}) $ vs iteration.
- Comment on convergence, oscillations, and divergence.

```
# Plotting theta histories for different etas on the same graph with
the x axis as iterations
plt.figure(figsize=(8, 5))
for i, eta in enumerate(etas):
    plt.plot(theta_histories[i], label=f'eta={eta}')
plt.xlabel('Iterations')
```

```
plt.ylabel('Theta Value')
plt.title('Theta History for Different Step Sizes')
plt.legend()
plt.grid()
plt.show()
```




```
# Plotting the loss histories for different etas on the same graph
with the x axis as iterations
plt.figure(figsize=(8, 5))
for i, eta in enumerate(etas):
    plt.plot(loss_histories[i], label=f'eta={eta}')
plt.yscale('log')
plt.xlabel('Iterations')
plt.ylabel('log Loss L(theta)')
plt.title('Loss History for Different Step Sizes')
plt.legend()
plt.grid()
plt.show()
```



1.5 Relate your observations to the discussion in class about:

- step-size being too small / too large,
- the role of convexity,
- how the “just right” step size leads to fast convergence.

Exercise 2: Backtracking Line Search

Consider the non-convex function

$ L(\theta) = \theta^{4} - 3*\theta^{2} + 2 $

```
def l(theta):
    return theta**4 - 3*(theta**2) + 2

def grad_l(theta):
    return 4*(theta**3) - 6*theta
```

2.1 Implement Gradient Descent with Backtracking, using the Armijo condition, considering the backtracking(...) function from class.

```
def backtracking(L, grad_L, theta, eta0=1.0, beta=0.5, c=1e-4):
    """
```

```
    Return a step size eta that satisfies the Armijo condition:
        L(theta - eta*g) <= L(theta) - c * eta * ||g||^2
    """
    eta = eta0
    g = grad_L(theta)
    g_norm2 = np.dot(g, g)

    # se:   Loss_k+1 <= Loss_k - Armijo_constante * eta *
norma_2_gradiente
    # allora riduci eta
    while L(theta - eta * g) > L(theta) - c * eta * g_norm2:
        eta *= beta
    return eta

def GD_backtracking(l, grad_l, theta_0, maxit, eta, tolL, tolTheta):
    loss_history = [l(theta_0)]
    theta_history = [theta_0]

    # GD step
    for k in range(maxit):
        # compute step size via backtracking
        eta = backtracking(l, grad_l, theta_0, eta0=eta)

        theta = theta_0 - eta * grad_l(theta_0)

        if(np.linalg.norm(grad_l(theta))<tolL or
(np.linalg.norm(theta-theta_0)<tolTheta)):
            break

        loss_history.append(l(theta))
        theta_history.append(theta_0)

        theta_0 = theta
    return theta, k, loss_history, theta_history
```

2.2 Test different initial points theta=[-2 , 0.5, 2]

```
starting_thetas = [-2.0, 0.5, 2.0]

maxit = 100
tolL = 1e-6
tolTheta = 1e-6
eta = 0.5 # high initial step size because backtracking will reduce it
as needed

loss_histories = []
theta_histories = []

for theta_0 in starting_thetas:
    theta_opt, num_iter, loss_history, theta_history =
```

```
GD_backtracking(l, grad_l, theta_0, maxit, eta, tolL, tolTheta)
    loss_histories.append(loss_history)
    theta_histories.append(theta_history)
    print("Starting theta:", theta_0, " Optimal theta:", theta_opt, "
iterations:" , num_iter, " L(theta_opt):", l(theta_opt))
```

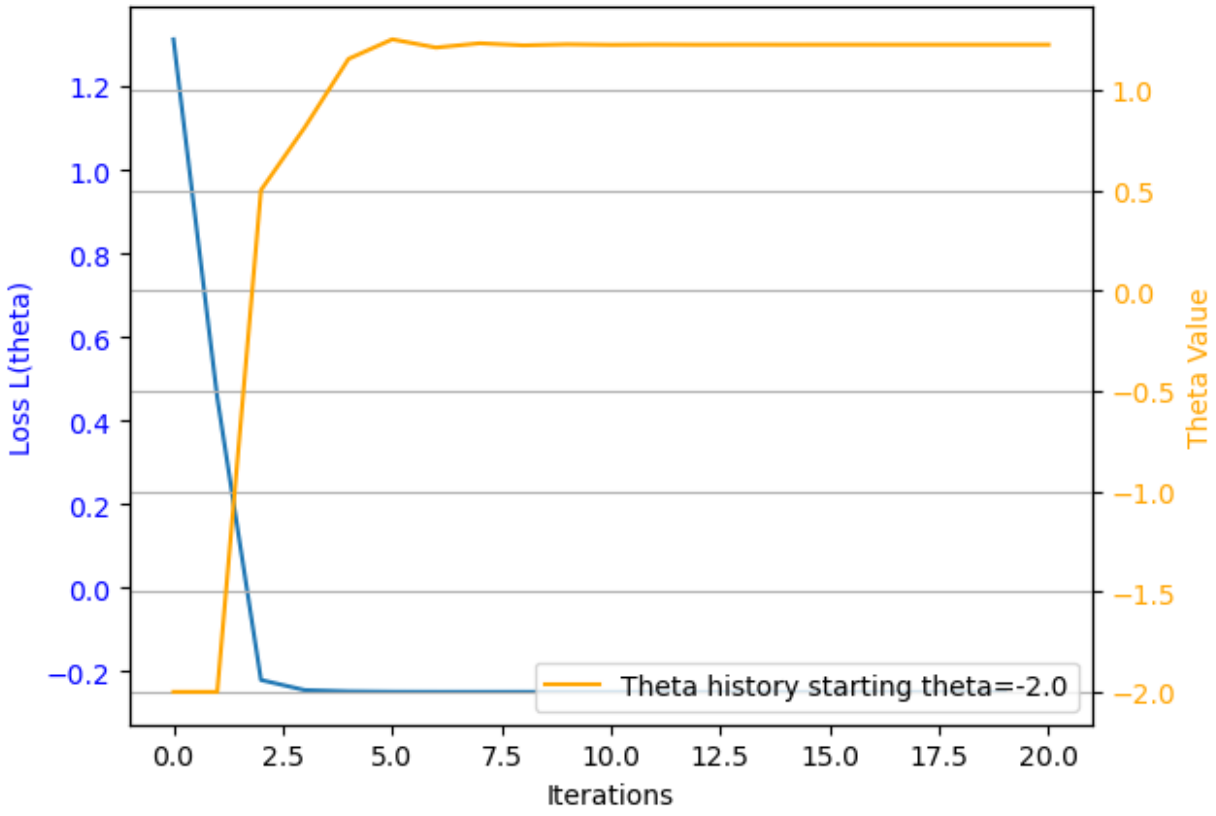
```
Starting theta: -2.0  Optimal theta: 1.2247446583384174  iterations:
20  L(theta_opt): -0.24999999999972733
Starting theta: 0.5  Optimal theta: 1.2247446103586532  iterations: 18
L(theta_opt): -0.249999999999591
Starting theta: 2.0  Optimal theta: -1.2247446583384174  iterations:
20  L(theta_opt): -0.24999999999972733
```

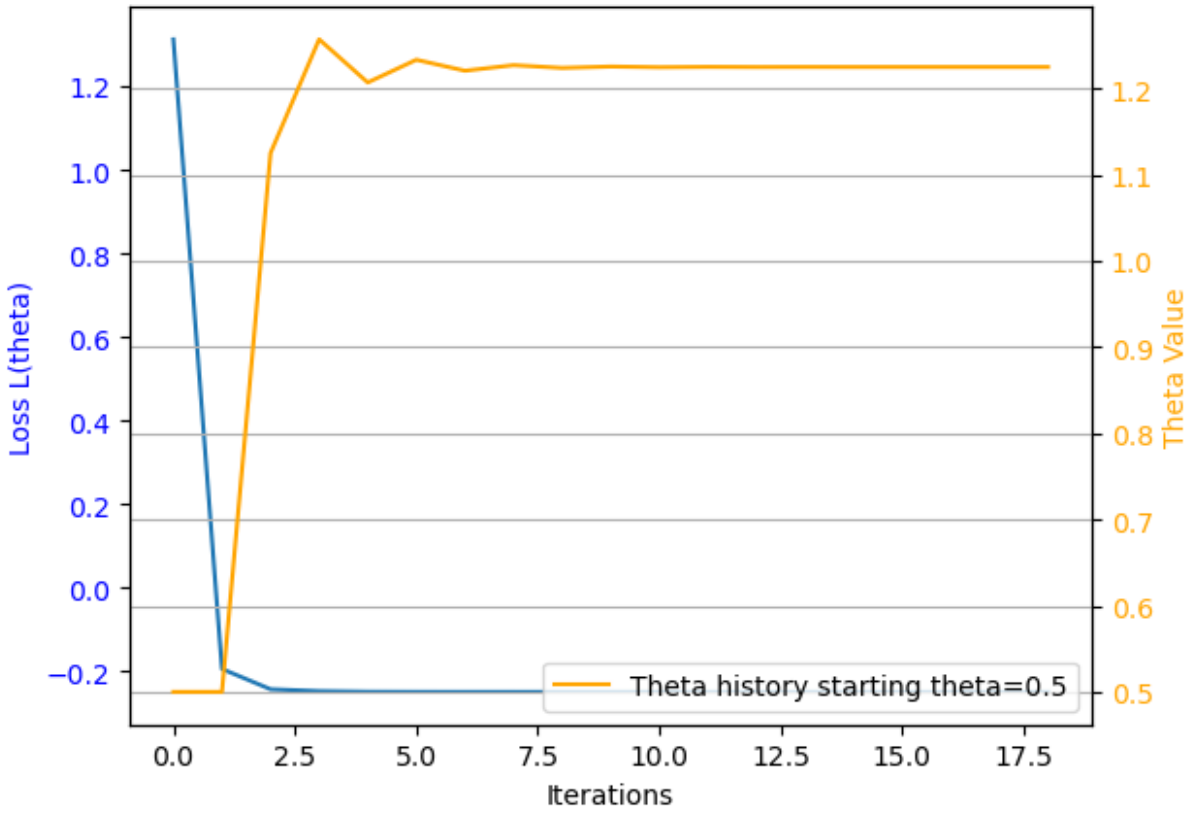
2.3 For each starting point, plot:

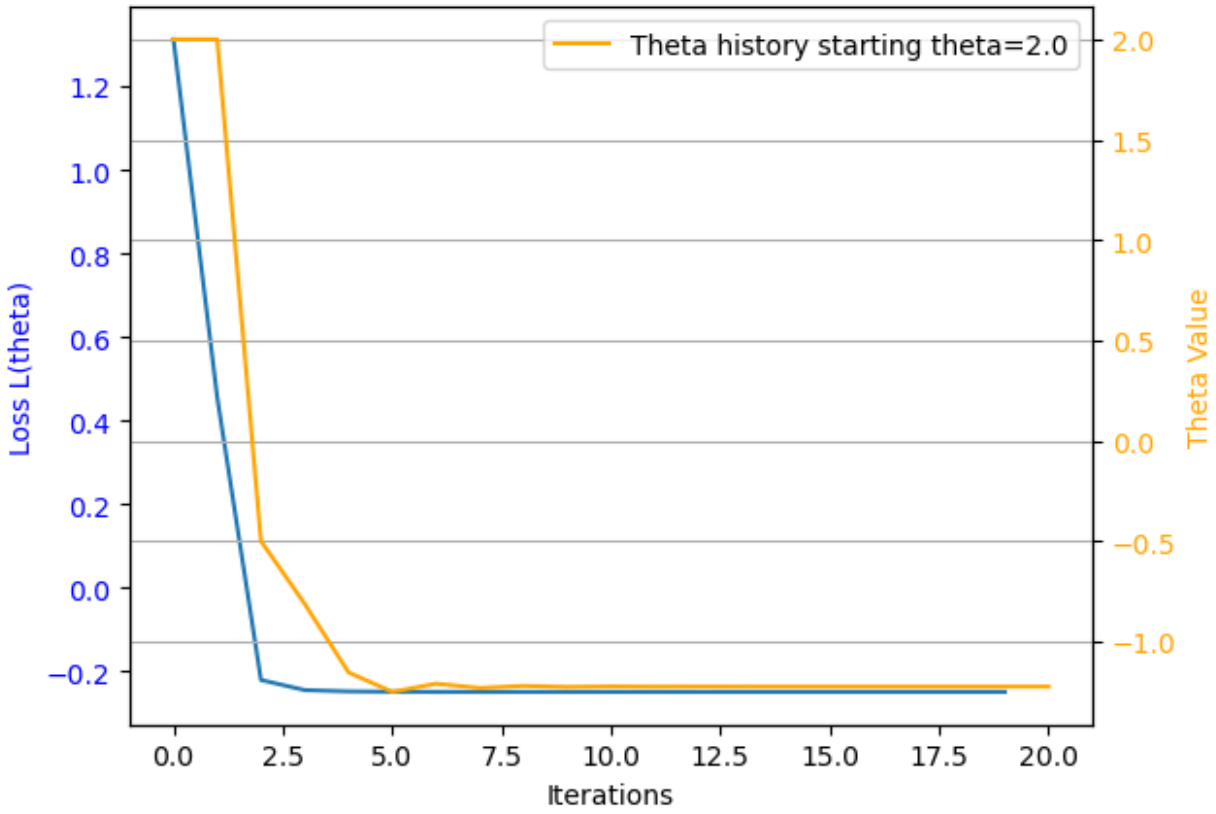
- the function curve L(theta) in 1D in the domain [-3, 3]
- the trajectory of the iterates theta overlaid on the curve.

```
plt.figure(figsize=(8, 5))
for i, loss_history in enumerate(loss_histories):
    # array con solo i valori in [-3, 3]
    filter_loss_in_range = [loss for loss in loss_history if -3 <=
loss <= 3]
    fig, ax1 = plt.subplots()
    ax1.set_xlabel('Iterations')
    ax1.set_ylabel('Loss L(theta)', color='blue')
    ax1.plot(filter_loss_in_range, label=f'Starting
theta={starting_thetas[i]}')
    ax1.tick_params(axis='y', labelcolor='blue')

    ax2 = ax1.twinx()
    ax2.set_ylabel('Theta Value', color='orange')
    ax2.plot(theta_histories[i], color='orange', label=f'Theta history
starting theta={starting_thetas[i]}')
    ax2.tick_params(axis='y', labelcolor='orange')

    plt.legend()
    plt.grid()

plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```


Loss L(theta)
Theta Value
Iterations
Theta history starting theta=-2.0
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Loss L(theta)
Theta Value
Iterations
Theta history starting theta=0.5
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
1.2
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5



2.4 Discuss:

- Why different initializations converge to different minima.
- How backtracking automatically chooses a suitable step size at each iteration.
- Situations where constant step size would fail.

Exercise 3: GD in 2D

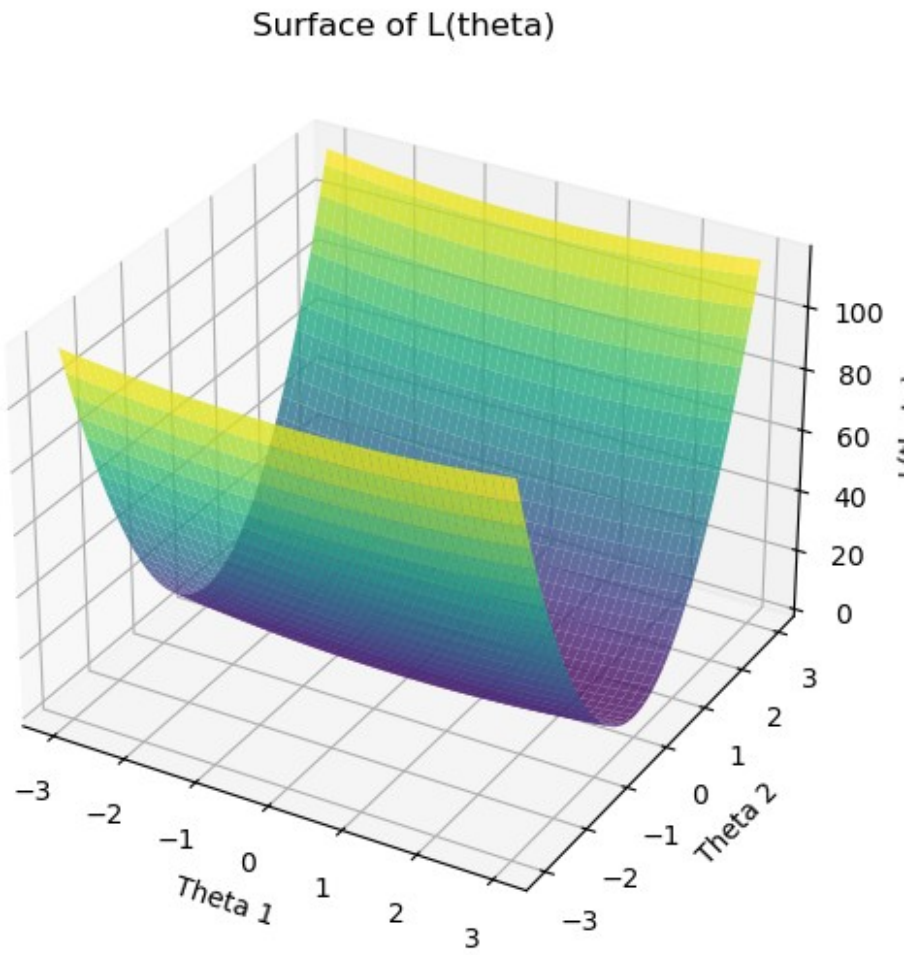
Consider the quadratic function:

$ L(\theta) = 1/2 * \theta^{T}A\theta $

$ A=\begin{pmatrix}a_{11} & a_{12}\a_{21} & a_{22}\end{pmatrix} $

```
A = np.array([[1.0, 0.0],
              [0.0, 25.0]])

def l(theta):
    return 1/2 * theta @ A @ theta.T


def grad_l(theta):
    return A @ theta
```

```
theta1_vals = np.linspace(-3, 3, 100)
theta2_vals = np.linspace(-3, 3, 100)
Theta1, Theta2 = np.meshgrid(theta1_vals, theta2_vals)
L_vals = 0.5 * (A[0,0]*Theta1**2 + A[1,1]*Theta2**2)

# Plot tree-dimensional surface of L(theta)
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(Theta1, Theta2, L_vals, cmap='viridis', alpha=0.8)
ax.set_xlabel('Theta 1')
ax.set_ylabel('Theta 2')
ax.set_zlabel('L(theta)')
ax.set_title('Surface of L(theta)')
plt.show()
```



3.1 Implement Gradient Descent in 2D, as seen in class.

3.2 Plot the level sets of L (using the helper code provided in the lecture notes) and overlay the iterates $ \theta^{k} $ for:

- $\eta = 0.02$

- $\eta = 0.05$ (borderline),
- $\eta = 0.1$ (diverging).

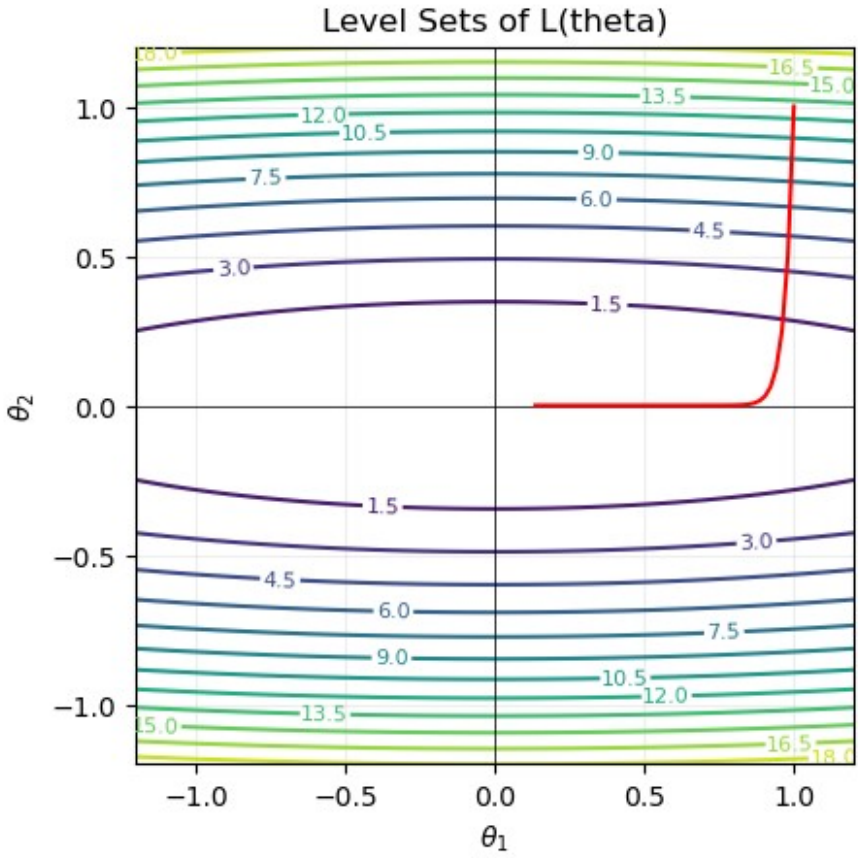
To overlay the iterates on the plot, simply add a plt.plot(...) function taking as input the coordinates $ \theta_{1}^{k} $ and $ \theta_{2}^{k} $ for each k

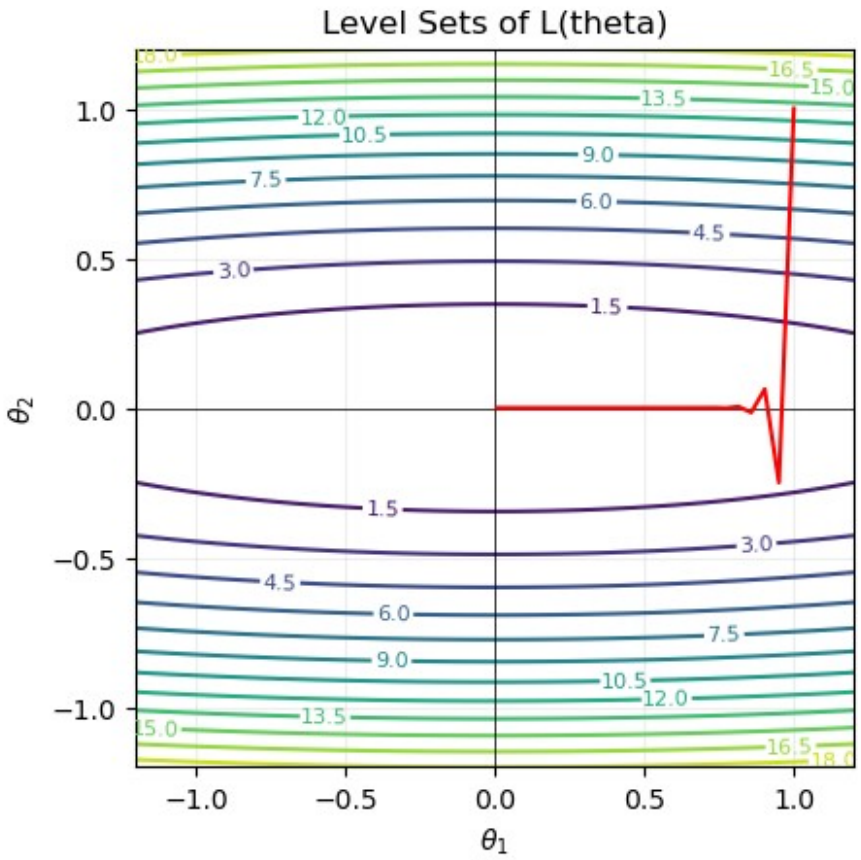
```
theta_0 = np.array([1.0, 1.0])

maxit = 100
tolL = 1e-6
tolTheta = 1e-6
etas = np.array([0.02, 0.05, 0.1])

loss_histories = []
theta_histories = []

for eta in etas:
    theta_0 = np.array([1.0, 1.0])
    theta_opt, num_iter, loss_history, theta_history = GD(l, grad_l,
theta_0, maxit, eta, tolL, tolTheta)
    loss_histories.append(loss_history)
    theta_histories.append(theta_history)
    print(f"Eta: {eta} => Optimal theta: {theta_opt}, iterations:
{num_iter}, L(theta_opt): {l(theta_opt)}")
```

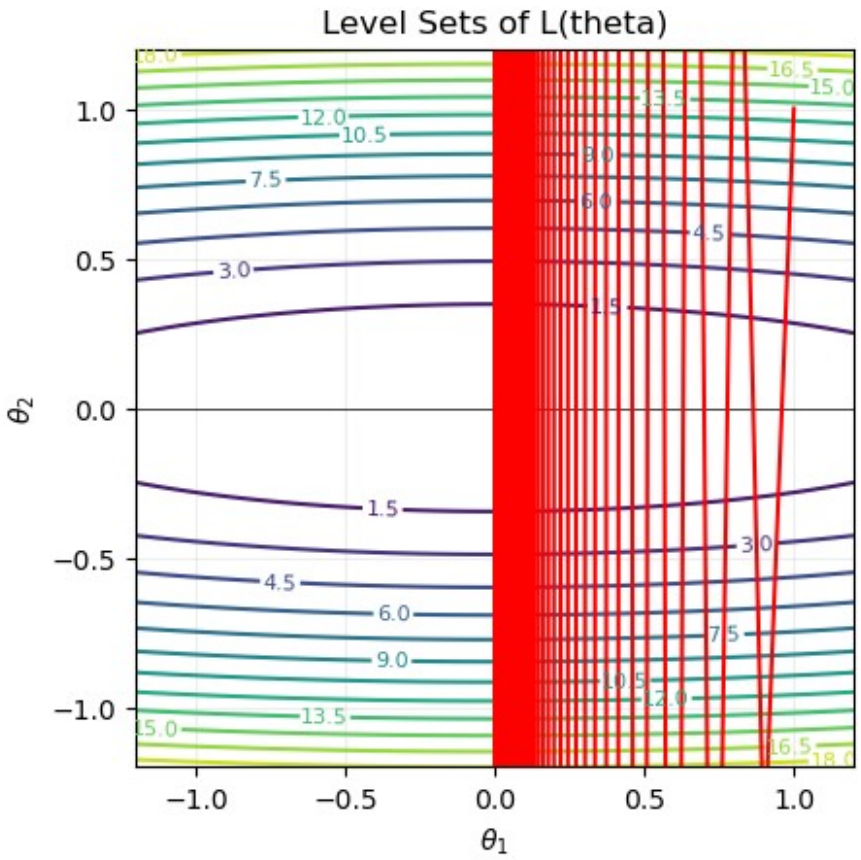
```
Eta: 0.02 => Optimal theta: [1.32619556e-01 7.88860905e-31],
iterations: 99, L(theta_opt): 0.008793973302860778
Eta: 0.05 => Optimal theta: [5.92052922e-03 6.22301528e-61],
iterations: 99, L(theta_opt): 1.752633312441451e-05
Eta: 0.1 => Optimal theta: [2.65613989e-05 4.06561178e+17],
iterations: 99, L(theta_opt): 2.0661498884852898e+36
```

```
def quad_levelsets(A, xlim=(-3,3), ylim=(-3,3), ngrid=400,
                   ncontours=12, title=None):
    xs = np.linspace(xlim[0], xlim[1], ngrid)
    ys = np.linspace(ylim[0], ylim[1], ngrid)
    X, Y = np.meshgrid(xs, ys)
    Z = 0.5*(A[0,0]*X**2 + 2*A[0,1]*X*Y + A[1,1]*Y**2)  # theta^T A
theta, left-multiplied convention
    cs = plt.contour(X, Y, Z, levels=ncontours)
    plt.clabel(cs, inline=True, fontsize=8)
    plt.axhline(0, lw=0.5, color='k')
    plt.axvline(0, lw=0.5, color='k')
    plt.gca().set_aspect('equal', 'box')
    if title:
        plt.title(title)
    plt.xlabel(r'$\theta_1$')
    plt.ylabel(r'$\theta_2$')
    plt.grid(alpha=0.2)
```

```
    # ensure axis bounds are set so subsequent plotting is clipped to
these limits
    plt.xlim(xlim)
    plt.ylim(ylim)

for theta_history in theta_histories:
    x = np.array(theta_history)
    quad_levelsets(A, xlim=(-1.2,1.2), ylim=(-1.2,1.2), title='Level
Sets of L(theta)')
    plt.plot(x[:,0], x[:,1], color='red')
    plt.show()
```




Level Sets of L(theta)
16.5
15.0
13.5
12.0
10.5
9.0
7.5
6.0
4.5
3.0
1.5
1.5
3.0
4.5
6.0
7.5
9.0
10.5
12.0
13.5
15.0
16.5
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0
θ_2
θ_1



3.3 Comment on:

- the elongated ellipses produced by ill-conditioning,
- the gradient direction compared to the level set lines,
- zig-zag behaviour for large condition numbers,
- the relation to the lecture discussion on slow convergence in narrow valleys.

Es 4 Exact Line Search vs Backtracking

Ricerca Lineare Esatta (Exact Line Search)

$$\eta^{¿} = arg\min_{\eta>0} L\left(\theta_k + \eta\, d_k\right)$$

$$\nabla L\left(\theta_k + \eta\, d_k\right)^T d_k = 0$$

let:

$ L(\theta) = 1/2 * \theta^{T} A \theta $

$ A=\begin{pmatrix}5 & 0\0 & 2\end{pmatrix} $

4.1 For GD with exact line search i.e. selecting the optimal step, you get:

$$\eta_k^{i} = \frac{g_k^T g_k}{g_k^T A g_k}$$

$ gk = A \theta^{k} $

```
A = np.array([[5.0, 0.0],
              [0.0, 2.0]])

def l(theta):
    return 1/2 * theta @ A @ theta.T

def grad_l(theta):
    return A @ theta

def GD_exact_line_search(l, grad_l, theta_0, maxit, tolL, tolTheta):
    loss_history = []
    theta_history = []

    for k in range(maxit):
        g = grad_l(theta_0)
        # exact line search step size
        eta = (g @ g) / (g @ A @ g)

        theta = theta_0 - eta * g

        if(np.linalg.norm(grad_l(theta))<tolL or
(np.linalg.norm(theta-theta_0)<tolTheta)):
            break

        loss_history.append(l(theta))
        theta_history.append(theta_0)

        theta_0 = theta
    return theta, k, loss_history, theta_history
```

4.2 For GD with backtracking, use the implementation provided in your notes.

```
def backtracking(L, grad_L, theta, eta0=1.0, beta=0.5, c=1e-4):
    eta = eta0
    g = grad_L(theta)
    g_norm2 = np.dot(g, g)

    # se:   Loss_k+1 > Loss_k - Armijo_constante * eta *
norma_2_gradiente
    # allora riduci eta
    while L(theta - eta * g) > L(theta) - c * eta * g_norm2:
        eta *= beta
    return eta
```

```
def GD_backtracking(l, grad_l, theta_0, maxit, tolL, tolTheta,
eta0=1.0):
    loss_history = []
    theta_history = []

    # GD step
    for k in range(maxit):
        # compute step size via backtracking
        eta = backtracking(l, grad_l, theta_0, eta0=eta0)

        theta = theta_0 - eta * grad_l(theta_0)

        if(np.linalg.norm(grad_l(theta))<tolL or
(np.linalg.norm(theta-theta_0)<tolTheta)):
            break

        loss_history.append(l(theta))
        theta_history.append(theta_0)

        theta_0 = theta
    return theta, k, loss_history, theta_history
```

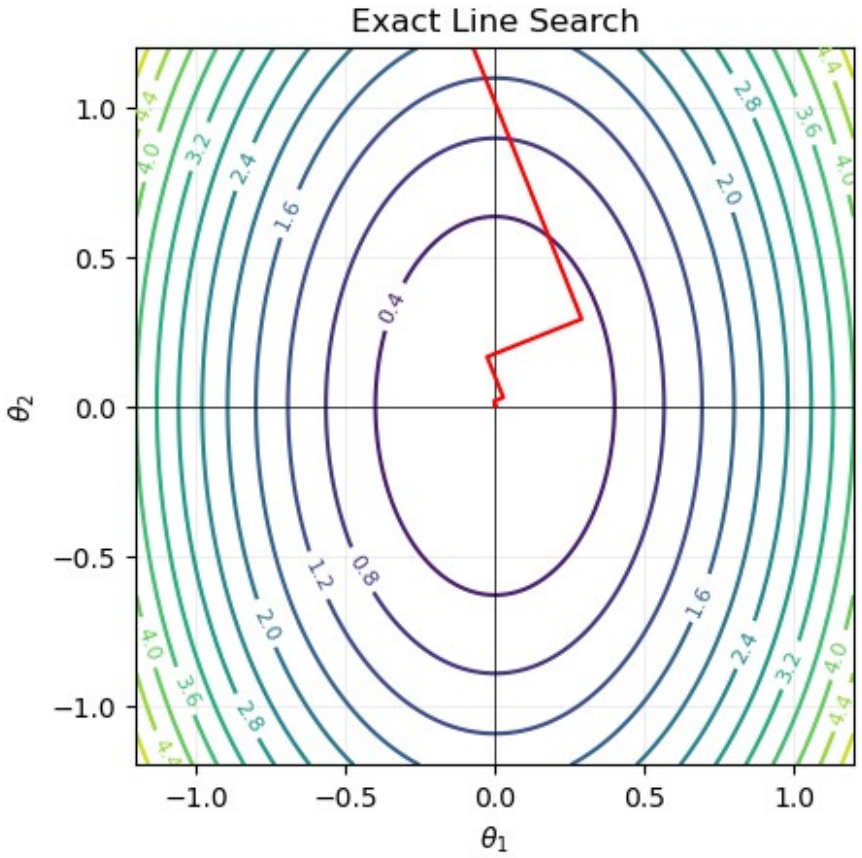
4.3 Starting from $ \theta_{0} = (3,3)^{T}
: $-Plottrajectoriesonthelevel-setmap.-PlotthelossL¿θ^{k})vs iteration for both methods.

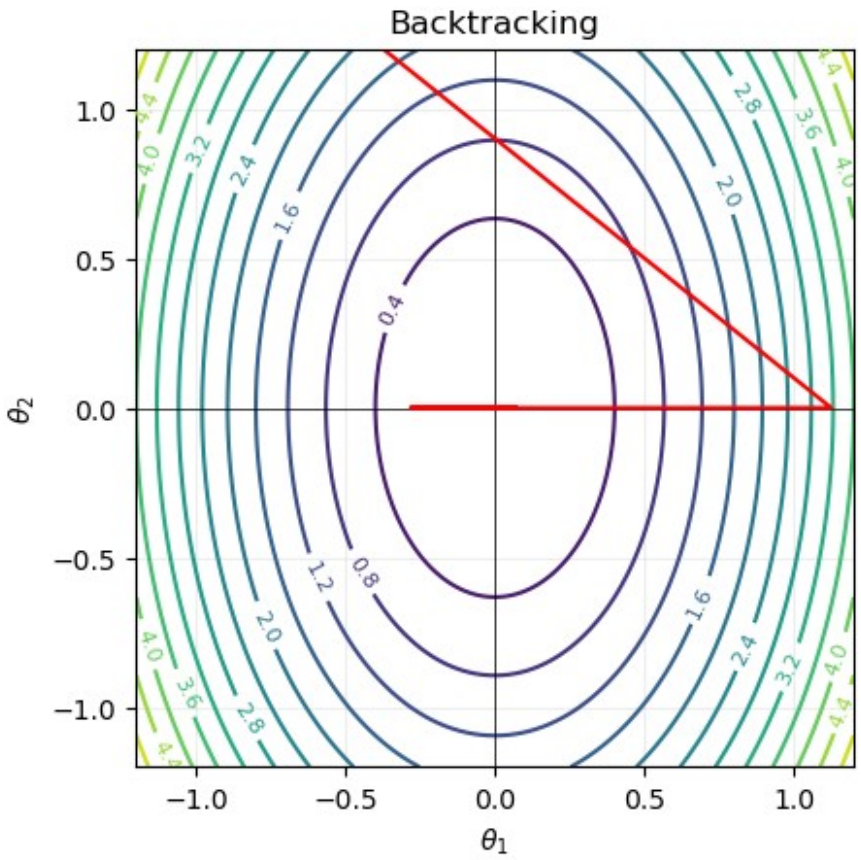
```
theta_0 = np.array([3.0, 3.0])

maxit = 100
tolL = 1e-6
tolTheta = 1e-6

theta_opt_1, num_iter_1, loss_history_1, theta_history_1 =
GD_exact_line_search(l, grad_l, theta_0.copy(), maxit, tolL, tolTheta)
theta_opt_2, num_iter_2, loss_history_2, theta_history_2 =
GD_backtracking(l, grad_l, theta_0.copy(), maxit, tolL, tolTheta)

print("Exact Line Search: Optimal theta:", theta_opt_1, " iterations:"
, num_iter_1,  "Loss: ", l(theta_opt_1))
print("Backtracking: Optimal theta:", theta_opt_2, " iterations:" ,
num_iter_2, "Loss: ", l(theta_opt_2))
```

```
Exact Line Search: Optimal theta: [-2.13550533e-08  1.33469083e-07]
iterations: 14 Loss:  1.8954091841682185e-14
Backtracking: Optimal theta: [6.70552254e-08 0.00000000e+00]
iterations: 13 Loss:  1.124100812432971e-14
```

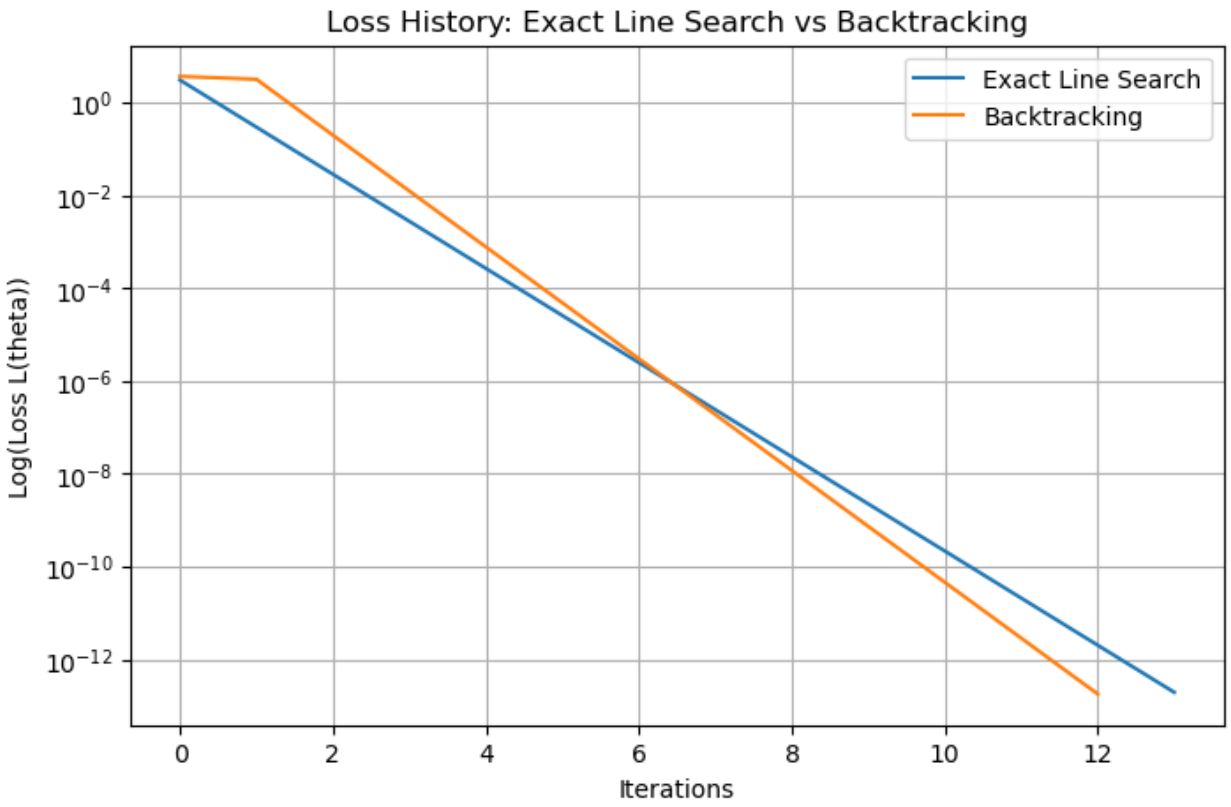
```
x = np.array(theta_history_1)
quad_levelsets(A, xlim=(-1.2,1.2), ylim=(-1.2,1.2), title='Exact Line
```

```
Search')
plt.plot(x[:,0], x[:,1], color='red')
plt.show()

x = np.array(theta_history_2)
quad_levelsets(A, xlim=(-1.2,1.2), ylim=(-1.2,1.2),
title='Backtracking')
plt.plot(x[:,0], x[:,1], color='red')
plt.show()
```






```
plt.figure(figsize=(8, 5))
plt.plot(loss_history_1, label='Exact Line Search')
plt.plot(loss_history_2, label='Backtracking')
plt.legend()
plt.yscale('log')
plt.xlabel('Iterations')
plt.ylabel('Log(Loss L(theta))')
plt.title('Loss History: Exact Line Search vs Backtracking')
plt.grid()
plt.show()
```



4.4 Compare:

- speed of convergence,
- smoothness of step-sizes.

Exercise 5: Gradient Descent on the Rosenbrock Function

The 2-dimensional Rosenbrock function is defined as:

$$(\Theta)=(1-\Theta_1)^2+100(\Theta_2-\Theta_1^2)^2,$$

with a unique global minimum at:

$$\Theta^{\prime}=(1,1), L(\Theta_1^{\prime},\Theta_2 y^{\prime})=0.$$

Despite having only one minimum, its landscape is **extremely challenging**:

- The valley is long and curved.
- The condition number varies drastically across the region.
- Gradients can point almost orthogonally to the valley direction.
- Gradient Descent may zig-zag and make very slow progress toward the solution.

This makes Rosenbrock a perfect testbed to study the difficulties of pure Gradient Descent.

1. Implement the function and its gradient to run the Gradient Descent algorithm. Test its performance using both **constant step size** and the **backtracking algorithm**.

2. Plot the level sets of L: use the standard 2D contour plotting approach practiced earlier.
   Ensure that you plot:

- A wide range (e.g. $\Theta_1 \in [-2,2), \Theta_2 \in [-1,3)$)
- A reasonable number of contour lines to visualize the narrow valley.

1. Choose multiple initial points, testing at least the following four:

- $\Theta^{(0)}=(-1.5,2)$
- $\Theta^{(0)}=(-1,0)$
- $\Theta^{(0)}=(0,2)$
- $\Theta^{(0)}=(1.5,1.5)$

Run both:

- GD with constant step size (try $\eta=10^{-3}, 10^{-4}, 10^{-5}$),
- GD with backtracking line search.

```
# The 2-dimensional Rosenbrock function is defined as:
def rosenbrock(theta):
    return (1 - theta[0])**2 + 100 * (theta[1] - theta[0]**2)**2

def grad_rosenbrock(theta):
    dL_dtheta1 = -2 * (1 - theta[0]) - 400 * theta[0] * (theta[1] -
theta[0]**2)
    dL_dtheta2 = 200 * (theta[1] - theta[0]**2)
    return np.array([dL_dtheta1, dL_dtheta2])

# Initial thetas
Initial_thetas = np.array([
    [-1.5, 2.0],
    [-1.0, 0.0],
    [0, 2],
    [1.5, 1.5]
])

etas = [0.001, 0.0001, 0.00001]


configuratitons = []
for theta_0 in Initial_thetas:
    for eta in etas :
        configuratitons.append({'eta': eta, 'theta_0': theta_0})

results_classic_gd = []
```

```
for configuration in configuratitons:
    maxit = 1000
    tolL = 1e-6
    tolTheta = 1e-6
    eta = configuration['eta']
    theta_0 = configuration['theta_0']
    theta_opt, num_iter, loss_history, theta_history = GD(rosenbrock,
grad_rosenbrock, theta_0, maxit, eta, tolL, tolTheta)
    results_classic_gd.append((theta_0, eta, theta_opt, num_iter,
rosenbrock(theta_opt), loss_history, theta_history))

print("Results with Classic Gradient Descent:\n")
print(f"{'theta_0':>12} | {'eta':>10} | {'theta_opt':>20} |
{'iter':>6} | {'L(theta_opt)':>12}")
for t0, eta, topt, it, loss, _, _ in results_classic_gd:
    print(f"{str(t0):>12} | {eta:10.6f} | {str(np.round(topt,4)):>20}
| {it:6d} | {loss:12.6g}")


results_backtracking_gd = []
for theta_0 in Initial_thetas:
    maxit = 1000
    tolL = 1e-6
    tolTheta = 1e-6
    eta0 = 1.0
    theta_opt, num_iter, loss_history, theta_history =
GD_backtracking(rosenbrock, grad_rosenbrock, theta_0, maxit, tolL,
tolTheta,  eta0=eta0)
    results_backtracking_gd.append((theta_0, theta_opt, num_iter,
rosenbrock(theta_opt), loss_history, theta_history))

print("\nResults with Backtracking Line Search:\n")
print(f"{'theta_0':>12} | {'theta_opt':>20} | {'iter':>6} |
{'L(theta_opt)':>12}")
for t0, topt, it, loss, _, _ in results_backtracking_gd:
    print(f"{str(t0):>12} | {str(np.round(topt,4)):>20} | {it:6d} |
{loss:12.6g}")
```

```
Results with Classic Gradient Descent:

     theta_0 |        eta |            theta_opt |   iter |
L(theta_opt)
 [-1.5  2. ] |   0.001000 |    [-0.6603  0.444 ] |    999 |
2.76291
 [-1.5  2. ] |   0.000100 |    [-1.3664  1.8748] |    999 |
5.60592
 [-1.5  2. ] |   0.000010 |    [-1.4157  2.0119] |    999 |
5.8415
   [-1.  0.] |   0.001000 |      [0.5979 0.3555] |    999 |
0.16207
```

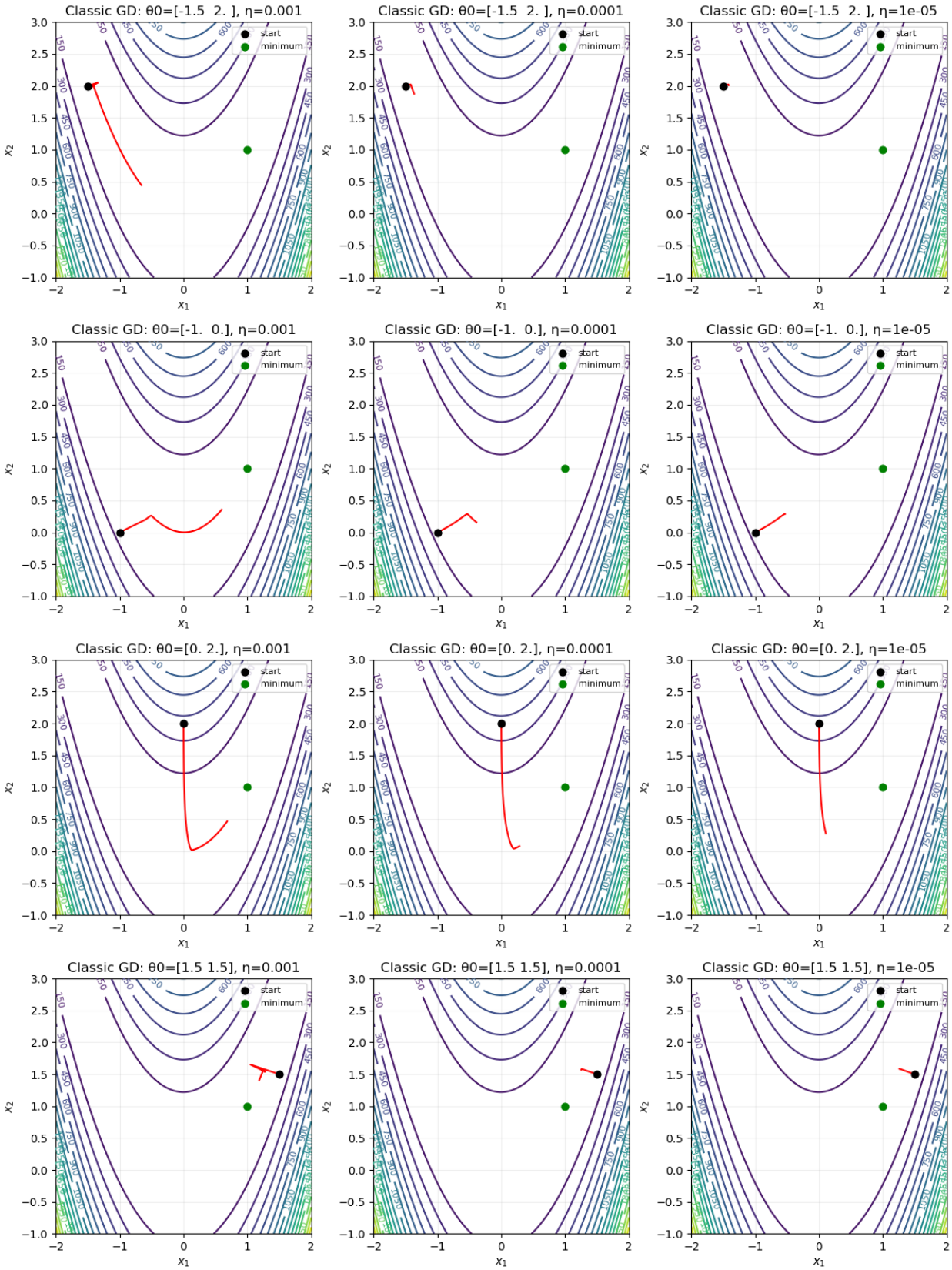
```
   [-1.  0.] |   0.000100 |    [-0.3878  0.1572] |    999 |
1.93063
   [-1.  0.] |   0.000010 |    [-0.5341  0.2866] |    999 |
2.35353
     [0. 2.] |   0.001000 |      [0.6837 0.466 ] |    999 |
0.100266
     [0. 2.] |   0.000100 |      [0.2845 0.0779] |    999 |
0.512907
     [0. 2.] |   0.000010 |      [0.1075 0.2743] |    999 |
7.70163
   [1.5 1.5] |   0.001000 |      [1.1841 1.4026] |    999 |
0.0339181
   [1.5 1.5] |   0.000100 |      [1.2523 1.5691] |    999 |
0.0637268
   [1.5 1.5] |   0.000010 |      [1.2588 1.5855] |    999 |
0.0670533

Results with Backtracking Line Search:

     theta_0 |            theta_opt |   iter | L(theta_opt)
 [-1.5  2. ] |      [1.5657 2.4548] |    999 |     0.321181
   [-1.  0.] |      [0.9412 0.8855] |    999 |   0.00347331
     [0. 2.] |      [0.9199 0.8464] |    999 |    0.0064135
   [1.5 1.5] |      [1.1337 1.2854] |    999 |    0.0178689
```

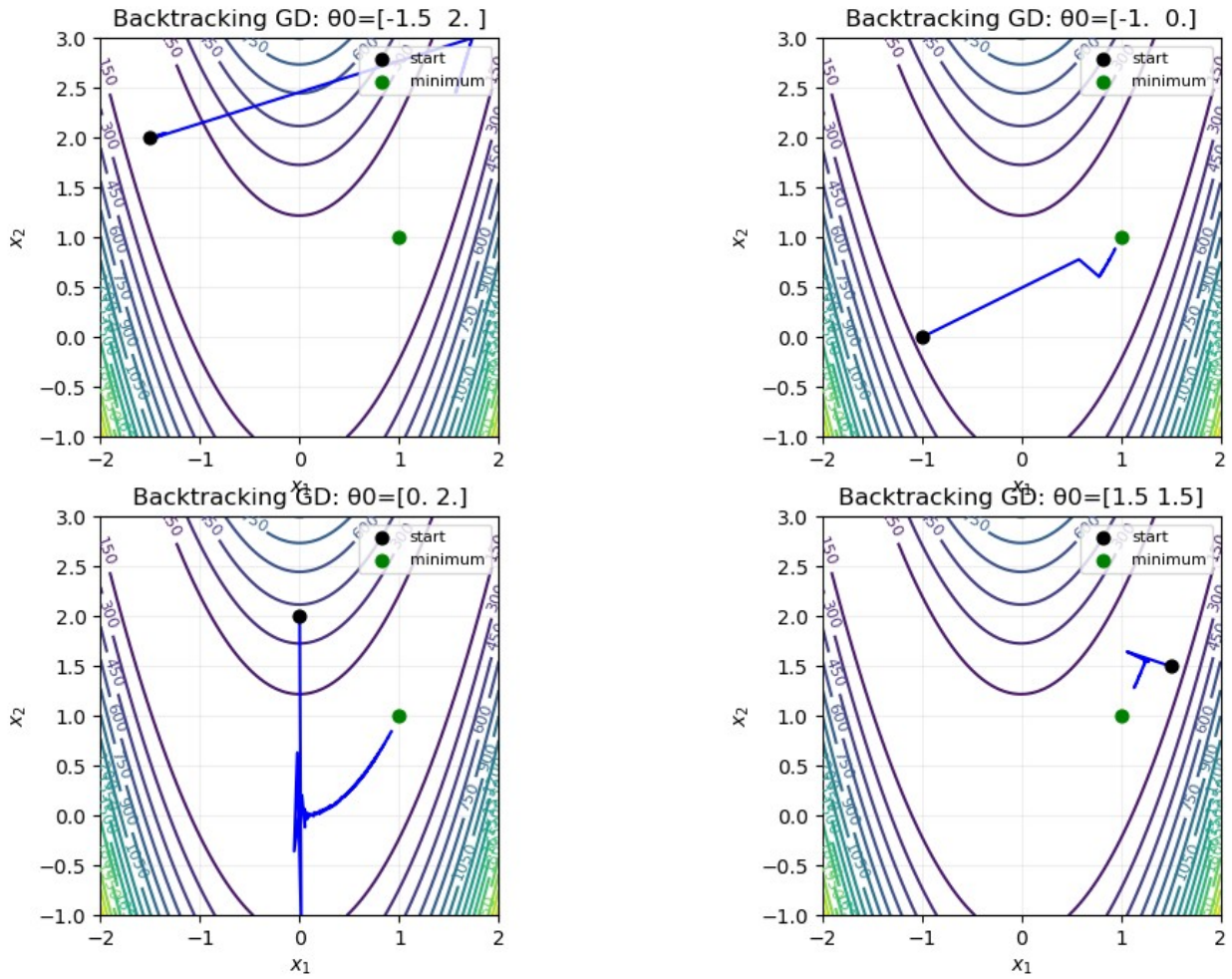
1. Visualize and analyze the trajectories: For each initialization and each training method:
    a. Plot the sequence of iterates $\left(\Theta_1^{(k)}, \Theta_2^{(k)}\right)$ as a path on the level-set map.
    b. Plot the loss curve $L\left(\Theta^{(k)}\right)$ vs iteration.
    c. Observe and explain:
        - whether the method enters the valley,
        - how long it takes to "turn" correctly along the valley direction,
        - whether the method zig-zags,
        - whether step sizes become too small (with constant η) or too large (divergence).

```python
# Plotting level line of Rosenbrock function using
def rosenbrock_levelsets(xlim=(-2,2), ylim=(-1,3), ngrid=400,
ncontours=20, title=None):
    xs = np.linspace(xlim[0], xlim[1], ngrid)
    ys = np.linspace(ylim[0], ylim[1], ngrid)
    X, Y = np.meshgrid(xs, ys)
    Z = (1 - X)**2 + 100 * (Y - X**2)**2
    cs = plt.contour(X, Y, Z, levels=ncontours)
    plt.clabel(cs, inline=True, fontsize=8)
    plt.axhline(0, lw=0.5, color='k')
    plt.axvline(0, lw=0.5, color='k')
```

```
    plt.gca().set_aspect('equal', 'box')
    if title:
        plt.title(title)
    plt.xlabel(r'$x_1$')
    plt.ylabel(r'$x_2$')
    plt.grid(alpha=0.2)
    # ensure axis bounds are set so subsequent plotting is clipped to
these limits
    plt.xlim(xlim)
    plt.ylim(ylim)

# print the theta history for every configuratiton of classic GD
n_plots = len(configuratitons)
n_cols = 3
n_rows = int(np.ceil(n_plots / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(12, 4*n_rows))
axes = axes.flatten()
for ax, configuration, result_classic_gd in zip(axes, configuratitons,
results_classic_gd):
    theta_0 = configuration['theta_0']
    eta = configuration['eta']
    theta_history = result_classic_gd[6]
    x = np.array(theta_history)
    # draw level sets on the given axis
    xs = np.linspace(-2, 2, 400)
    ys = np.linspace(-1, 3, 400)
    X, Y = np.meshgrid(xs, ys)
    Z = (1 - X)**2 + 100 * (Y - X**2)**2
    cs = ax.contour(X, Y, Z, levels=20)
    ax.clabel(cs, inline=True, fontsize=8)
    ax.plot(x[:,0], x[:,1], color='red')
    ax.scatter(theta_0[0], theta_0[1], color='black', s=40,
marker='o', zorder=5, label='start')
    ax.scatter(1, 1, color='green', s=40, marker='o', zorder=5,
label='minimum')
    ax.legend(loc='upper right', fontsize=8)
    ax.set_title(f'Classic GD: θ0={theta_0}, η={eta}')
    ax.set_xlabel(r'$x_1$')
    ax.set_ylabel(r'$x_2$')
    ax.grid(alpha=0.2)
    ax.set_aspect('equal', 'box')
    ax.set_xlim(-2, 2)
    ax.set_ylim(-1, 3)
# nascondi eventuali assi vuoti
for ax in axes[n_plots:]:
    ax.axis('off')
plt.tight_layout()
plt.show()
```

```
# print the theta history for every configuratiton of backtracking GD
n_plots = len(Initial_thetas)
n_cols = 2
n_rows = int(np.ceil(n_plots / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(12, 4*n_rows))
axes = axes.flatten()
for ax, configuration, result_backtracking_gd in zip(axes,
Initial_thetas, results_backtracking_gd):
    theta_0 = configuration
    theta_history = result_backtracking_gd[5]
    x = np.array(theta_history)
    # draw level sets on the given axis
    xs = np.linspace(-2, 2, 400)
    ys = np.linspace(-1, 3, 400)
    X, Y = np.meshgrid(xs, ys)
    Z = (1 - X)**2 + 100 * (Y - X**2)**2
    cs = ax.contour(X, Y, Z, levels=20)
    ax.clabel(cs, inline=True, fontsize=8)
    ax.plot(x[:,0], x[:,1], color='blue')
    ax.scatter(theta_0[0], theta_0[1], color='black', s=40,
marker='o', zorder=5, label='start')
    ax.scatter(1, 1, color='green', s=40, marker='o', zorder=5,
label='minimum')
    ax.legend(loc='upper right', fontsize=8)
    ax.set_title(f'Backtracking GD: θ0={theta_0}')
    ax.set_xlabel(r'$x_1$')
    ax.set_ylabel(r'$x_2$')
    ax.grid(alpha=0.2)
    ax.set_aspect('equal', 'box')
    ax.set_xlim(-2, 2)
    ax.set_ylim(-1, 3)
```


Classic GD: θ0=[-1.5 2.], η=0.001
Classic GD: θ0=[-1.5 2.], η=0.0001
Classic GD: θ0=[-1.5 2.], η=1e-05
Classic GD: θ0=[-1. 0.], η=0.001
Classic GD: θ0=[-1. 0.], η=0.0001
Classic GD: θ0=[-1. 0.], η=1e-05
Classic GD: θ0=[0. 2.], η=0.001
Classic GD: θ0=[0. 2.], η=0.0001
Classic GD: θ0=[0. 2.], η=1e-05
Classic GD: θ0=[1.5 1.5], η=0.001
Classic GD: θ0=[1.5 1.5], η=0.0001
Classic GD: θ0=[1.5 1.5], η=1e-05
start
minimum
x_1
x_2




```
n_plots = len(Initial_thetas)
n_cols = 2
n_rows = int(np.ceil(n_plots / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(12, 4*n_rows))
axes = axes.flatten()

for idx, (ax, theta_0, result_backtracking_gd) in enumerate(zip(axes,
Initial_thetas, results_backtracking_gd)):
    # Plot all Classic GD runs for this theta_0 (different etas)
    for result_classic_gd in results_classic_gd:
        if np.array_equal(result_classic_gd[0], theta_0):
            eta = result_classic_gd[1]
            loss_history_classic = result_classic_gd[5]
            ax.plot(loss_history_classic, label=f'Classic GD η={eta}',
linestyle='--')

    # Plot Backtracking GD for this theta_0
    loss_history_backtracking = result_backtracking_gd[4]
    ax.plot(loss_history_backtracking, label='Backtracking GD',
```

```
color='blue', linewidth=2)

    ax.set_yscale('log')
    ax.set_xlabel('Iterations')
    ax.set_ylabel('Log(Loss L(theta))')
    ax.set_title(f'Loss History: θ0={theta_0}')
    ax.legend()
    ax.grid()

# nascondi eventuali assi vuoti
for ax in axes[n_plots:]:
    ax.axis('off')

plt.tight_layout()
plt.show()
```